

Desarrollar el Diagrama de Clases con sus atributos, relaciones, cardinalidades y métodos correspondiente del siguiente problema planteado: Desarrollamos un software de MiniMarket donde se deben dejar registrados los movimientos de Stock cuando se realiza una venta a los clientes o una compra a los proveedores de un producto determinado, por ejemplo “Gaseosa en botella”. Identificar los requisitos “Funcionales” y los requisitos “No Funcionales”, además de realizar el prototipado de Interfaz de

Usuario del problema planteado utilizando una herramienta seleccionada por el equipo para tal fin.

Requisitos funcionales:

- Crear factura de venta
- Crear factura de compra
- Modificar automáticamente el stock con compras o ventas
- Hacer pedido a proveedor cuando el stock llega a un umbral
- Mostrar historial de movimientos de stock
- Registrar proveedor

Requisitos no funcionales:

- Velocidad: las operaciones de registro y consulta no deben tardar más de 5 segundos
- Seguridad: solo empleados autorizados pueden registrar proveedores, ver el historial del stock (administradores)
- Integridad de los datos
- Actualización inmediata del stock
- Interfaz de usuario clara
- Sistema disponible durante todo el tiempo que el minimarket atienda

Al problema planteado en el ejercicio, proponer nuevas funcionalidades que permitan la existencia de las relaciones UML de Asociación, Agregación, Composición y Herencia.

- Asociación: la asociación ya está presente, por ejemplo entre la clase FacturaDeVenta y Cliente, ya que una factura de venta se le hace a un cliente en particular y en la factura debe figurar dicho cliente.
- Composición: ya existe en el UML entre Detalle y Factura. Una factura está compuesta por detalles de la venta, por otro lado si esa factura desapareciera del sistema, no tiene sentido seguir guardando esos detalles, ya que pertenecían únicamente a esa factura.
- Herencia: existe entre la clase padre Factura y sus hijas FacturaDeVenta y FacturaDeCompra. Como ambas son facturas, hay características y comportamientos en común que se detallen en Factura, sin embargo queremos distinguir entre las facturas que el minimarket le hace a los clientes y las que los

proveedores le hacen al minimarket, por lo que necesitamos dos clases distintas y ambas heredan de Factura.

- Agregación: para que nuestro modelo también tenga agregación podríamos agregar una funcionalidad de armar un combo de productos con descuento. Esto le permitiría al minimarket registrar en el sistema combos de productos con un determinado precio. Así habría agregación entre la clase Combo (que tendría atributos precio, validoDesde y validoHasta) y Producto, ya que un combo está compuesto de productos que no se eliminan del sistema aunque el combo ya no se venda más.